



Unidad 4: Estructura de Datos

Introducción a la Programación y Computación 2

Escuela de Ingeniería en Ciencias y Sistemas

Facultad de Ingeniería

Universidad de San Carlos de Guatemala

Agenda

Recordatorios

- Ficheros
- Diccionarios
- Tuplas
- Expresiones Regulares
- Buenas Prácticas

Recursos



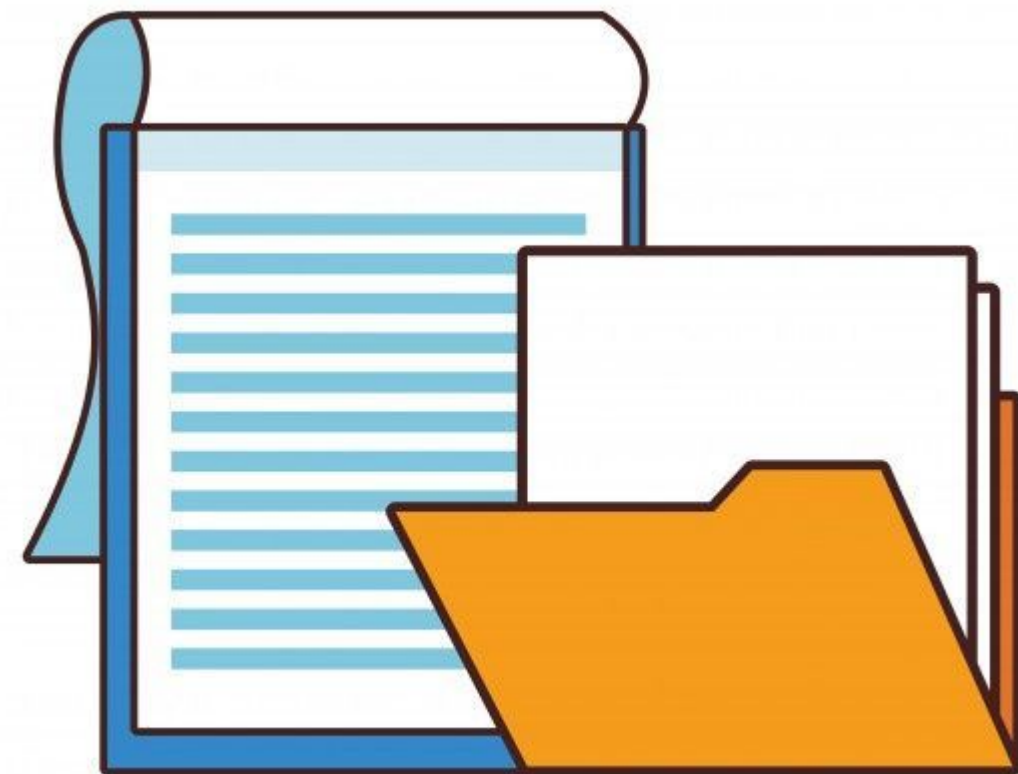
Competencias que Desarrollaremos

Aplica tuplas, listas, diccionarios nativos de Python en programas funcionales para almacenar y manipular conjunto de datos.



Ficheros

Un fichero es un conjunto de datos relacionados entre sí, que son almacenados de forma permanente en dispositivos tales como discos duros, memorias flash, etc. y pueden ser tratados, hasta cierto punto, como una unidad.



Fichero de texto

Un fichero de texto puede ser considerado una secuencia de líneas, de igual modo que una cadena en Python puede ser considerada una secuencia de caracteres.

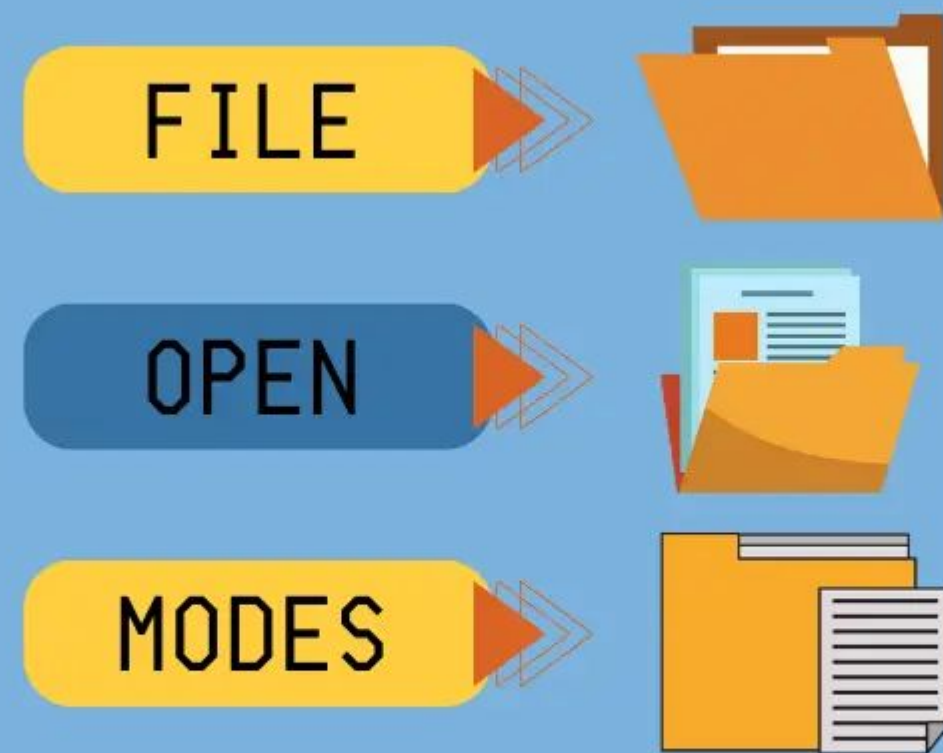


Apertura de ficheros

Cuando se desea leer o escribir en un archivo (nos referimos en el disco duro) primero debemos abrir el fichero. Al abrir el fichero nos comunicamos con el sistema operativo, que sabe dónde se encuentran almacenados los datos de cada archivo. Cuando se abre un fichero, se está pidiendo al sistema operativo que lo busque por su nombre y se asegure de que existe. En este ejemplo, abrimos el fichero **mbox.txt**, que debería estar guardado en la misma carpeta en la que te encontrabas cuando iniciaste Python.

```
>>> manf = open('mbox.txt')  
>>> print manf  
<open file 'mbox.txt', mode 'r' at 0x1005088b0>
```


Modos para abrir un fichero



r

Solo lectura. El fichero solo se puede leer. Es el modo por defecto si nos se indica.

w

Sólo escritura. En el fichero solo se puede escribir. Si ya existe el fichero, machaca su contenido.

a

Adición. En e fichero solo se puede escribir. Si ya existe el fichero, todo lo que se escriba se añadirá al final del mismo.

x

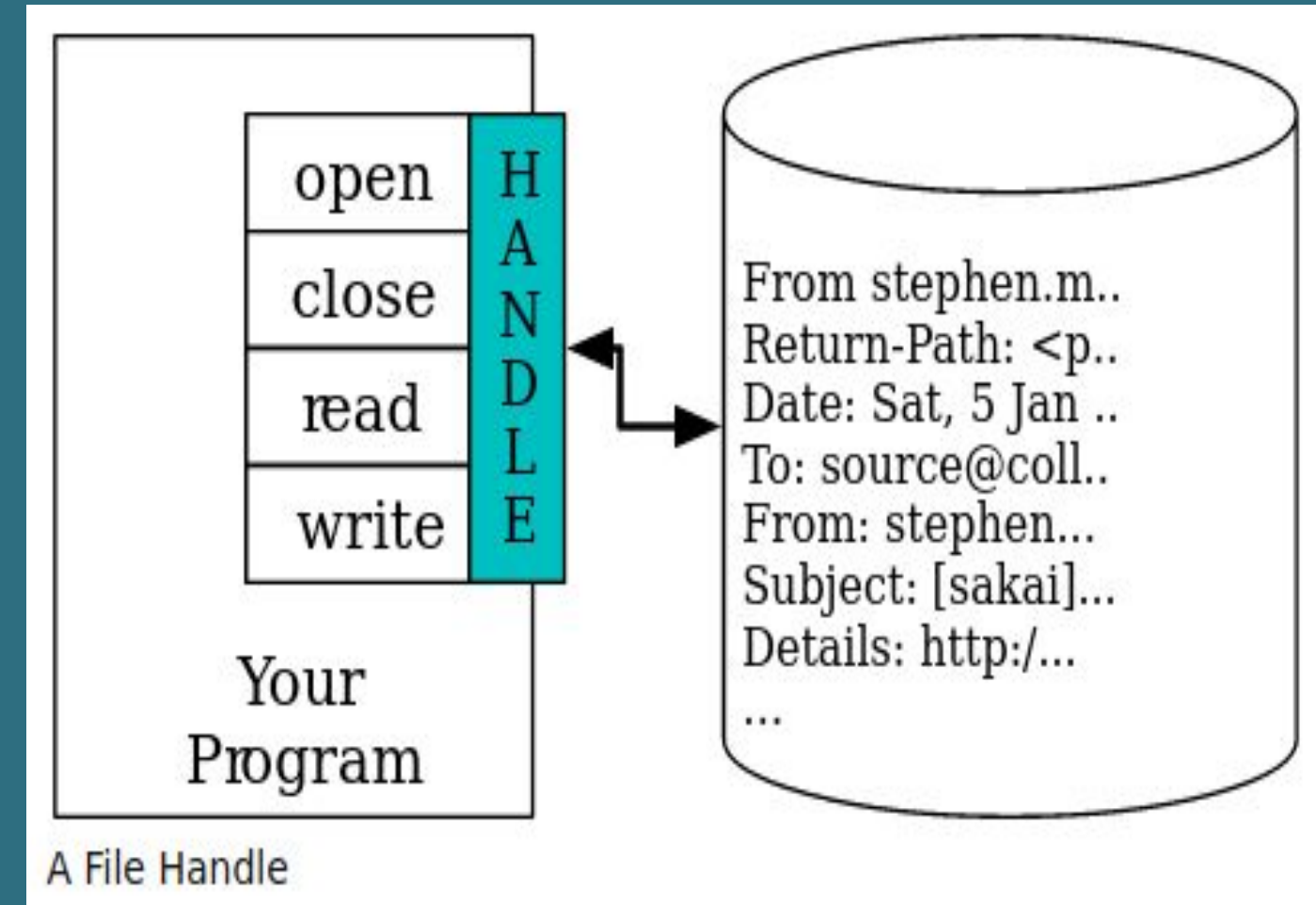
como 'w', pero si existe el fichero lanza una excepción.

r+

Lectura y escritura. El fichero se puede leer y escribir.

Apertura de ficheros

Si la apertura tiene éxito, el sistema operativo nos devuelve un manejador de fichero (file handle). El manejador de fichero no son los datos que contiene en realidad el archivo, sino que se trata de un “manejador” (handle) que se puede utilizar para leer esos datos.



Apertura de ficheros

Si el fichero no existe, open fallará con un traceback y no obtendrás ningún manejador para poder acceder a su contenido:

```
>>> manf = open('cosas.txt')
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
IOError: [Errno 2] No such file or directory: 'cosas.txt'
```

Lectura de

Ficheros

Con el método `read()` es posible leer un número de bytes determinados. Si no se indica número se leerá todo lo que reste o si se alcanzó el final del fichero devolverá una cadena vacía.

```
with open('readdemo.txt', 'r') as f:  
    print(f.read())  
    f.write("Reading fresh")
```

Escritura de Ficheros

El método `write()` escribe una cadena y el método `writelines()` escribe una lista a un archivo. Si en el momento de escribir el archivo no existe, se creará uno nuevo.

```
text = "This is new content"
# writing new content to the file
fp = open("write_demo.txt", 'w')
fp.write(text)
print('Done Writing')
fp.close()

# Open the file for reading the new contents
fp = open("write_demo.txt", 'r')
print(fp.read())
fp.close()
```

Apendice de Ficheros

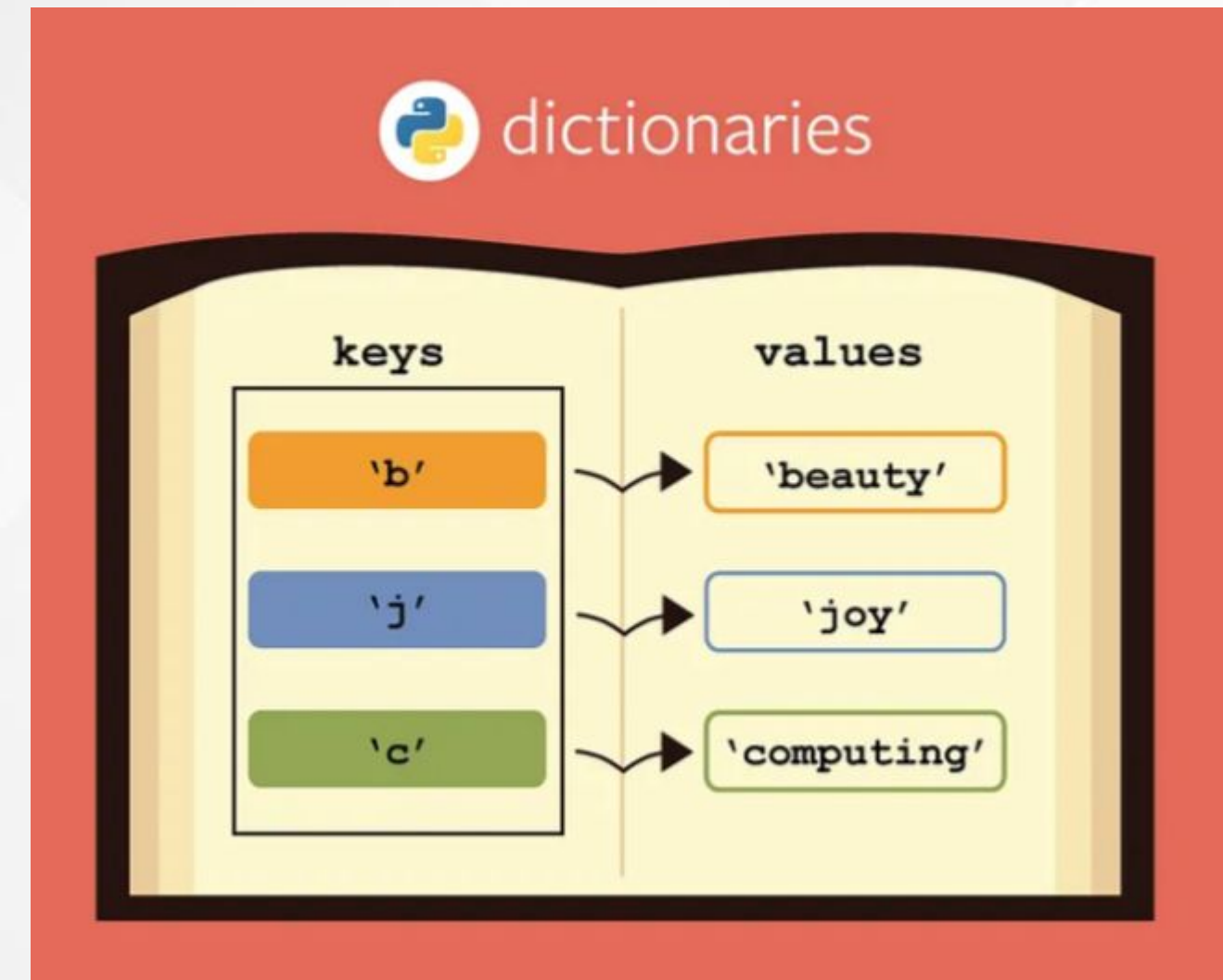
Se refiere a abrir el archivo en un modo que permite agregar datos al final del archivo existente, sin sobrescribir su contenido.

```
# Open and Append at Last
fp = open("sample2.txt", "a")
fp.write(" Added this line by opening the file in append mode ")

# Opening the file again to read
fp = open("sample2.txt", "r")
print(fp.read())
fp.close()
```

Diccionario

En Python un diccionario es un conjunto desordenado de pares clave-valor, en el que la clave debe ser única y puede ser cualquier tipo de datos inmutable como enteros, cadenas, booleanos, etc. Para los valores que se pueden tener dentro de un diccionario no hay restricciones ya que pueden ser de cualquier tipo como listas, cadenas, enteros, tuplas, incluso otro diccionario.



Sintaxis de los Diccionarios

Cada elemento esta integrado por dos elemento
separados por dos puntos (:). Clave (key) y Valor

Dict_Name = { key1 : value1 , key2 : value2 , key3 : value3 }

Elementos separados por comas

Llaves

Aplicar lo aprendido (Ejemplo)



Conceptos claves

aprendidos

- Ficheros
- Modos de un fichero
- Manipulación de un fichero en Python.
- Diccionarios
- Tuplas
- Expresiones regulares



Valores

Responsabilidad:

- Leer y escribir ficheros implica el manejo de datos potencialmente sensibles.
- El programador debe ser responsable al abrir, modificar y guardar información.
- **Ejemplo:** Al guardar registros de usuarios, se debe evitar sobrescribir o perder información accidentalmente.



Valores

Orden y Organización:

- Los diccionarios permiten mapear claves y valores de forma clara.
- Fomentan el pensamiento lógico y estructurado.
- **Ejemplo:** En una agenda digital, cada nombre se relaciona ordenadamente con su número de teléfono.



Valores

Integridad:

- Las tuplas son inmutables: su contenido no cambia, igual que los principios éticos.
- **Ejemplo:** Cuando defines coordenadas geográficas, estas no deben alterarse sin justificación.



Valores

Presición:

- Las “regex” requieren exactitud para funcionar correctamente.
- **Ejemplo:** Una expresión regular mal escrita puede validar mal contraseñas o permitir errores de seguridad.





GRACIAS POR SU
ATENCIÓN!
DUDAS

RECUERDA QUE TENEMOS NUESTRO FORO SEMANAL DONDE PUEDES
CONSULTAR CUALQUIER DUDA QUE TE SURJA EN LA SEMANA